COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/21/26	**DATE SUBMITTED:**	10/21/26
NAME:	Francis Elijah J. Formalejo Rayven Ferry C. Licas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13 **Parallel Algorithms and Multithreading**

1. PROCEDURE OUTPUT

What procedures did you perform?

Strategy	Algorithm	Analysis
Recursion	Binary	**Formalejo:** Calls its own function passing updated start or end indices to shrink the search space on each pass **Licas:** Reduces the search area step by step by calling the function again with updated sub-array bounds until finding the target.
Brute Force	Linear	**Formalejo:** It cans elements repeatedly from start to end without skipping, checking every location until a match is found **Licas:** Checks every single slot in order starting from index zero until reaching the end or locating the target value
Backtracking	Depth Search	**Formalejo:** Traverses deep down a branch until reaching a dead end, then reverses to the last decision point

		to try another path **Licas:** It goes all the way down one path until it hits a dead end, then backs up to try another way
Greedy	Selection	**Formalejo:** It picks the smallest available item from the unsorted list at each step then it changes the previous swaps **Licas:** It looks for the smallest number left in the list and swaps it right into place
Divide-and-Conquer	Merge Sort	**Formalejo:** Splits an array into smaller halves repeatedly, then combines the solved sub lists to build a fully sorted result **Licas:** It splits a big list into smaller halves, sorts them, and then puts them back together into one list

Memoization:

Screenshot	```#include <iostream>
	#include <thread>
	#include <string>
	
	void print(int n, const std::string &str) {
	    std::cout << "Printing integer: " << n << std::endl;
	    std::cout << "Printing string: " << str << std::endl;```

	``
	}
	
	int main() {
	    std::thread t1(print, 10, "T.I.P.");
	    t1.join();
	    return 0;
	}
	``
	
	``
	Printing integer: 10
	Printing string: T.I.P.
	``

Analysis	**Formalejo:** The main program starts a helper thread to print the text, then it calls join so that the main program waits for the helper to finish. **Licas:** The code runs a secondary thread to handle printing the number and text. Next, it will be calling join() to force the primary program loop to pause until the child thread finishes.

Bottom up DP:

Screenshot	```
	#include <iostream>
	#include <thread>
	#include <vector>
	#include <string>
	
	void print(int n, const std::string &str) {
	    std::string msg = std::to_string(n) + " : " + str;
	    std::cout << msg << std::endl;
	}
	
	int main() {
	    std::vector<std::string> s = {
	        "T.I.P.",
	        "Competent",
	        "Computer",
	        "Engineers"
	    };
	
	    std::vector<std::thread> threads;
	``

```
    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
0 : T.I.P.3 : Engineers
1 : Competent
2 : Computer
```

Analysis	**Formalejo:** The program executes off four threads at once inside a loop. All the four threads run the print function at the exact same time. And I don't really know why the engineers is up there, tried redoing the code but its still the same. **Licas:** Theres in the code to handle printing at the same time. Because all four threads run at the same time without being in sync, their execution timing is random, causing the output order to print randomly rather than sequentially

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PART A:

1. Write a definition of multithreading and its advantages/disadvantages.
   Multithreading is a programming model that lets you do multiple execution paths or threads run simultaneously inside a single program process.

   Advantages:
   - Keeps applications responsive because background threads handle long tasks while user interface threads stay active
   - Enhances CPU utilization and execution speed on multi core processors by distributing tasks across hardware cores.
   - Saves computing resources and memory since threads within the same process share memory and resources directly without expensive inter-process communication.

   Disadvantages:

- Increases coding complexity and risks race conditions where concurrent updates corrupt shared data.
- Introduces deadlocks where threads freeze indefinitely while waiting for locked resources held by each other.
- Creates debugging difficulties because non-deterministic thread execution makes timing bugs hard to reproduce.
- Adds CPU overhead from frequent context switching and memory allocation for thread stacks.

2. Rationalize the use of multithreading by providing at least 3 use-cases

    - Apps: Because like word processors or photo editing apps, it uses extra threads to save documents or export photos in the background so you can keep editing without the screen freezing.
    - Web Browsers: Because browsers run separate threads to download files, watch videos, and load web pages all at the same time without slowing down.
    - Video Games: Because game engines run different threads for background music, physics, and graphics, etc…, so that the game plays smoothly without lagging.

3. Differentiate between parallelism and concurrency

    Concurrency is about structures and handling many things at once. Like on a single computer core, concurrency works by switching between tasks very fast so they make progress during double times. It keeps the apps responsive while waiting for the external tasks.

    Parallelism is about execution speed and doing tasks at the exact same time. Parallelism also requires multiple CPU cores because each core runs a separate task at the same time to finish hard work faster.

https://www.geeksforgeeks.org/operating-systems/benefits-of-multithreading-in-operating-system/

https://www.multisoftvirtualacademy.com/blog/common-advantages-and-disadvantages-of-multithreading-in-java

https://www.designgurus.io/answers/detail/what-are-the-pros-and-cons-of-multithreading

https://aws.amazon.com/what-is/multithreading/

https://www.bing.com/ck/a?!&&p=68c3a82ccee74bb0aa8498455b68d32404adc9aed08cebe4595bdf23d424cee5JmltdHM9MTc5MjM2ODAwMA&ptn=3&ver=2&hsh=4&fclid=3e4738d1-f52d-60f0-1c62-2efaf4ed61ba&u=a1aHR0cHM6Ly93d3cuZnJlZWNvZGVjYW1wLm9yZy9uZXdzL2NvbmN1cnJlbmN5LXZzLXBhcmFsbGVsaXNtLXdoYXRzLXRoZS1kaWZmZXJlbmNlLWFuZC13aHktc2hvdWxkLXlvdS1jYXJlLw

https://www.bing.com/ck/a?!&&p=b38b7a8f6c99d498b995ef31e94e7052cbefc82cc482f04b1b8b4b8aa7996dd3JmltdHM9MTc5MjM2ODAwMA&ptn=3&ver=2&hsh=4&fclid=3e4738d1-f52d-60f0-1c62-2efaf4ed61ba&u=a1aHR0cHM6Ly93d3cuZ2Vla3Nmb3JnZWVrcy5vcmcvb3BlcmF0aW5nLXN5c3RlbXMvZGlmZmVyZW5jZS1iZXR3ZWVuLWNvbmN1cnJlbmN5LWFuZC1wYXJhbGxlbGlzbS8

PART B:

```
#include <iostream>
#include <thread>

int global_var = 0;

void add(int value) {
    global_var += value;
}

int main() {
    std::cout << "Global_var: " << global_var << std::endl;

    std::thread T1(add, 10);
    std::cout << "Before T1.join(): " << global_var << std::endl;
    T1.join();
    std::cout << "After T1.join(): " << global_var << std::endl;

    std::thread T2(add, 20);
    std::cout << "Aefore T2.join(): " << global_var << std::endl;
    T2.join();
    std::cout << "After T2.join(): " << global_var << std::endl;

    std::thread T3(add, 30);
    std::cout << "Before T3.join(): " << global_var << std::endl;
    T3.join();
    std::cout << "After T3.join(): " << global_var << std::endl;

    return 0;
}
```

```
Global_var: 0
Before T1.join(): 0
After T1.join(): 10
Aefore T2.join(): 10
After T2.join(): 30
Before T3.join(): 30
After T3.join(): 60
```

Analysis:
Formalejo:
First it starts with global_var set to 0, then printing it right after creating T1 it will show 0 because the main thread runs faster than the work thread. Then calling T1.join() forces the main thread to pause until T1 completes the addition, which updates global_var to 10. Then repeating this again and again for T2 and T3 shows the original values before each join call.

Licas:
The program starts by running 3 separate worker threads in order. Each thread adds a number to global_var, then the main program prints the variable before calling join to show the value before the addition completes. Calling join pauses the main program until the worker thread finishes.

PART C:

```
#include <iostream>
#include <thread>
#include <vector>

using namespace std;

void bubbleSort(vector<int>& arr, int start, int end) {
    for (int i = start; i < end; i++) {
        for (int j = start; j < end - (i - start); j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void merge(vector<int>& arr, int start, int mid, int end) {
    vector<int> left(arr.begin() + start, arr.begin() + mid + 1);
    vector<int> right(arr.begin() + mid + 1, arr.begin() + end + 1);
```

```
    int i = 0, j = 0, k = start;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }

    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
}

int main() {
    vector<int> arr = {64, 34, 25, 12, 22, 11, 90, 88, 76, 45};
    int n = arr.size();
    int mid = n / 2 - 1;

    cout << "Unsorted array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    thread t1(bubbleSort, ref(arr), 0, mid);
    thread t2(bubbleSort, ref(arr), mid + 1, n - 1);

    t1.join();
    t2.join();

    merge(arr, 0, mid, n - 1);

    cout << "Sorted array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}
```

```
Unsorted array: 64 34 25 12 22 11 90 88 76 45
Sorted array: 11 12 22 25 34 45 64 76 88 90
```

Analysis:

Formalejo:

The thread t1 sorts the left half from index 0 to 4 while thread t2 sorts the right half from index 5 to 9 at the same time. Then calling join on both threads so that both halves are fully sorted before the main thread merges them back into a single array.

Licas:

The code makes thread 1 handle sorting the left side of the array while thread 2 sorts the right side one after another. Then the main code waits for both threads using join() then calls merge() to combine the two sorted halves into a final sorted list

Conclusion:

Formalejo:

In this activity, I saw a clear understanding on how multithreading breaks programs into smaller pieces to boost performance (like how devs in game do to reduce lag due to high quality of graphics), keep applications responsive, and manage concurrent execution. I learned how C++ uses functions like join() to handle thread syncs, and keeping shared memory safe. In the procedure part, I setted up worker threads with the main program showing how pausing execution until threads finish prevents race conditions during global variable updates. In the supplementary part, it was done by cleanly stepping global_var from 0 up to 10, 30, and 60, and by running bubble sort across two separate array halves at the same time before merging them into one. Overall, I feel like I did decent in this one, I just had a hard time doing the supplementary because I couldn't figure out the problem because I couldn't run or execute my code, it wasn't sending any notification errors so I had to redo everything and redownload vscode (lol). And for improvements, typing speed, because I still don't remember where the placements of [] , {}, ‘,”, etc…, and it's costing me a couple of seconds because of that, anything else is decently good.

Licas:

In this activity, I learned how multithreading splits the code into worker threads to speed up and do multiple task all at once and reduce some unwanted lag. And some functions like join pause the main thread so the worker threads complete it safely. Joining threads kept global variable updates accurate and sorted some array halves before merging it. This was why setting it up took extra time due to VS Code terminal errors, we didn't really have enough time to manually check what's wrong with the VS Code so we had to delete it and fully install it again with all the extensions that was already in there, so it's like we repuzzled the puzzle type. And after doing that, we only had a short amount of time to finish the code since downloading took us so much time. Overall, this activity was enjoyable, especially the procedure ones because of the different strategies and algorithms for me.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.